МЕТОДИЧКА ДЛЯ СТУДЕНТОВ

Тема пары: повторение NumPy и начало Pandas

Google Colab • Python • анализ данных

Что внутри методички	Для чего это нужно
Повторение NumPy	Чтобы уверенно работать с массивами, индексами, срезами и быстрыми вычислениями
Введение в Pandas	Чтобы перейти от обычных массивов к таблицам, строкам, столбцам и фильтрации данных
Код с пояснениями	Чтобы понимать не только результат, но и логику каждой строки
Практика и задания	Чтобы закрепить тему руками, а не только посмотреть на примеры

Материал рассчитан на 2 пары по 80 минут и подходит для выполнения в Google Colab.

1. Цель занятия

- Повторить NumPy настолько плотно, чтобы студент без подсказки мог создать массив, обратиться к элементу, взять срез, посчитать сумму, среднее и выполнить простые векторные операции.
- Понять, зачем после NumPy нужен Pandas и почему для табличных данных удобнее использовать DataFrame, а не вложенные списки и словари.
- Научиться создавать таблицу, смотреть ее структуру, выбирать столбцы и строки, фильтровать записи, сортировать данные и делать базовые выводы.

2. Что студент должен уметь после пары

Навык	Что это означает на практике
NumPy массив	Создать одномерный и двумерный массив, определить shape, size и ndim
Индексация и срезы	Получить нужный элемент, строку, столбец или диапазон значений
Векторные операции	Прибавить число ко всему массиву, умножить массив, сложить два массива
Pandas Series / DataFrame	Понять разницу между одним столбцом и полноценной таблицей
Фильтрация и сортировка	Найти нужные записи и упорядочить их по значению столбца
Базовая аналитика	Посчитать среднее, максимум, минимум и группировку по категориям

3. Почему этот урок важен

NumPy и Pandas - это основа почти любого дальнейшего движения в сторону аналитики данных, машинного обучения и искусственного интеллекта. До нейросети и до модели всегда идет работа с данными.

Если студент не умеет открыть данные, понять их форму, выбрать нужную часть, посчитать метрику и сделать вывод, дальше ему будет трудно. Поэтому сегодняшний урок - не просто новая библиотека, а фундамент.

4. Организация работы в Google Colab

1. Каждый логический кусок кода пишется в отдельной ячейке. Не надо сразу вставлять огромный кодовый блок на 50 строк.
2. После запуска ячейки нужно обязательно смотреть на вывод и понимать, что именно изменилось.
3. Если код не отработал, не переписывайте весь ноутбук. Сначала определите, на какой строке возникает ошибка и что именно она означает.
4. Все примеры из этой методички лучше повторять по шагам: сначала импорт, затем создание объекта, затем вывод результата.

```
# Подключение библиотек
import numpy as np
import pandas as pd
```

5. Повторение NumPy

5.1 Что такое NumPy

NumPy - это библиотека для работы с массивами и числовыми вычислениями. Она нужна тогда, когда обычных списков Python уже недостаточно по скорости, удобству или масштабу.

В обычном списке Python можно хранить значения, но массовые операции над числами выполнять неудобно. NumPy позволяет применять одну операцию сразу ко всему набору данных и делает это быстро.

Обычный список Python	Массив NumPy
Подходит для базового хранения данных	Подходит для вычислений и числовой обработки
Операции часто нужно делать через цикл	Операции можно применять сразу ко всему массиву
Менее эффективен на больших числовых данных	Оптимизирован для математики и анализа данных
Удобен для знакомства с Python	Нужен для AI, аналитики, математики и работы с данными

5.2 Импорт библиотеки

```
import numpy as np
```

- import - команда подключения библиотеки.
- numpy - реальное имя библиотеки.
- as np - короткий псевдоним. Почти все примеры в интернете и в практике пишутся именно так.
- Когда вы видите np.array(...), это означает, что вызывается функция из NumPy.

5.3 Создание одномерного массива

```
arr = np.array([10, 20, 30, 40, 50])
print(arr)
print(type(arr))
```

В первой строке создается переменная arr. Внутрь np.array(...) передается обычный список, а NumPy превращает его в специальный массив.

print(arr) выводит содержимое массива на экран.

type(arr) показывает тип объекта. В результате вы увидите numpy.ndarray. Это сокращение от n-dimensional array, то есть многомерный массив.

5.4 Индексация и срезы

```
print(arr[0])
print(arr[2])
print(arr[-1])
print(arr[1:4])
print(arr[:3])
print(arr[2:])
```

Выражение	Что оно делает
arr[0]	Берет первый элемент массива
arr[2]	Берет третий элемент массива
arr[-1]	Берет последний элемент
arr[1:4]	Берет элементы с индекса 1 до индекса 3 включительно по факту вывода
arr[:3]	Берет элементы с начала до индекса 2
arr[2:]	Берет элементы с индекса 2 до конца

5.5 Форма массива: shape, size, ndim

```
print(arr.shape)
print(arr.size)
print(arr.ndim)
```

- shape показывает форму массива. Для одномерного массива из пяти элементов это будет (5,).
- size показывает общее количество элементов.
- ndim показывает число измерений. У обычного линейного массива ndim равно 1.

5.6 Двумерный массив

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix)
print(matrix.shape)
print(matrix.ndim)
print(matrix[0])
print(matrix[1, 2])
```

Здесь создается двумерный массив: 2 строки и 3 столбца.

matrix.shape вернет (2, 3), потому что у нас две строки и три столбца.

matrix[0] дает первую строку целиком.

matrix[1, 2] означает: вторая строка, третий столбец.

5.7 Специальные массивы

```
zeros = np.zeros((2, 3))
ones = np.ones((3, 2))
full = np.full((2, 2), 7)

print(zeros)
print(ones)
print(full)
```

- np.zeros((2, 3)) создает массив размером 2 на 3, заполненный нулями.

- np.ones((3, 2)) создает массив размером 3 на 2, заполненный единицами.
- np.full((2, 2), 7) создает массив 2 на 2, где все ячейки равны 7.

5.8 Диапазон значений через arange

```
a = np.arange(1, 11)
b = np.arange(0, 20, 2)

print(a)
print(b)
```

arange работает похоже на range, но возвращает NumPy-массив. В первом примере будут числа от 1 до 10, во втором - числа от 0 до 18 с шагом 2.

5.9 Векторные операции

```
arr = np.array([1, 2, 3, 4, 5])

print(arr + 10)
print(arr * 2)
print(arr - 1)
print(arr / 2)
```

Главная идея NumPy в том, что операция применяется не к одному числу, а сразу ко всему массиву.

Если вы пишете arr + 10, то десять прибавляется к каждому элементу. Это называется векторизацией и именно она делает код короче и быстрее.

5.10 Операции между двумя массивами

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)
print(a * b)
```

Сложение и умножение идут попарно: первый элемент с первым, второй со вторым, третий с третьим. Поэтому массивы должны быть совместимы по размерам.

5.11 Агрегатные функции в NumPy

```
nums = np.array([10, 20, 30, 40, 50])

print(nums.sum())
print(nums.mean())
print(nums.max())
print(nums.min())
```

Метод	Назначение
sum()	Сумма всех элементов
mean()	Среднее значение

max()	Максимальный элемент
min()	Минимальный элемент

5.12 Мини-практика по NumPy

5. Создайте массив из 6 чисел и выведите первый, последний и срез с 2 по 5 элемент.
6. Создайте массив от 1 до 12 и умножьте все элементы на 3.
7. Создайте двумерный массив 2 на 3 и выведите его форму.
8. Посчитайте сумму, среднее, максимум и минимум для массива из оценок.

6. Переход от NumPy к Pandas

NumPy отлично подходит для чисел, массивов, матриц и вычислений. Но реальные данные редко выглядят как просто набор чисел. Обычно это таблицы: имя, возраст, город, балл, посещаемость и так далее.

Когда данные становятся табличными, намного удобнее использовать Pandas. Он позволяет работать со строками, столбцами, фильтрацией, сортировкой и аналитикой почти так же удобно, как в Excel, но через Python-код.

NumPy	Pandas
Работа с массивами и матрицами	Работа с таблицами и наборами записей
Подходит для числовых вычислений	Подходит для анализа табличных данных
Главный объект: ndarray	Главные объекты: Series и DataFrame
Хорош для математики и подготовки данных	Хорош для чтения, фильтрации и анализа данных

7. Введение в Pandas

7.1 Импорт библиотеки

```
import pandas as pd
```

Как и в случае с NumPy, здесь используется короткое общепринятое имя. Все основные функции библиотеки будут вызываться через префикс pd.

7.2 Что такое Series

```
ages = pd.Series([17, 18, 19, 20])
print(ages)
print(type(ages))
```

- Series - это одномерная структура данных. Проще говоря, один столбец.
- Слева выводятся индексы, справа - значения.
- Series часто используется как часть DataFrame, но сам по себе тоже полезен.

7.3 Что такое DataFrame

```
data = {
    'name': ['Алина', 'Руслан', 'Диана'],
    'age': [17, 18, 19],
    'score': [85, 72, 91]
}

df = pd.DataFrame(data)
print(df)
```

Словарь data содержит ключи и списки значений. Каждый ключ становится названием столбца.

pd.DataFrame(data) превращает словарь в таблицу. Получается структура с колонками и строками.

DataFrame - это основной объект Pandas, с которым вы будете работать чаще всего.

7.4 Учебный датасет для практики

```
data = {
    'name': ['Алина', 'Руслан', 'Диана', 'Мадина', 'Ержан', 'Айгерим', 'Дамир',
'Нурсултан'],
    'age': [17, 18, 19, 18, 17, 20, 19, 18],
    'group': ['A', 'A', 'B', 'B', 'A', 'C', 'C', 'B'],
    'city': ['Алматы', 'Алматы', 'Астана', 'Астана', 'Шымкент', 'Алматы', 'Тараз',
'Алматы'],
    'score': [85, 72, 91, 88, 67, 95, 70, 83],
    'attendance': [90, 80, 95, 92, 75, 98, 85, 88]
}

df = pd.DataFrame(data)
print(df)
```

Эта таблица будет использоваться почти во всех следующих примерах. Она специально сделана простой и понятной: студенты, возраст, группа, город, балл и посещаемость.

7.5 Быстрый просмотр таблицы

```
print(df.head())
print(df.info())
print(df.shape)
print(df.columns)
```

Команда	Что показывает
df.head()	Первые 5 строк таблицы
df.info()	Сведения о столбцах, количестве строк и типах данных
df.shape	Размер таблицы: количество строк и столбцов
df.columns	Список названий столбцов

7.6 Выбор одного и нескольких столбцов

```
print(df['name'])
print(df[['name', 'score']])
```

- df['name'] - это выбор одного столбца. Результатом будет Series.
- df[['name', 'score']] - это выбор нескольких столбцов. Здесь внутри передается список, поэтому нужны двойные квадратные скобки.
- Разницу между одной и двумя скобками нужно запомнить сразу, потому что это одна из самых частых ошибок новичков.

7.7 Выбор строк через iloc

```
print(df.iloc[0])
print(df.iloc[0:3])
```

iloc выбирает строки по номеру позиции. Если написать df.iloc[0], будет первая строка. Если написать df.iloc[0:3], будут выбраны первые три строки.

7.8 Фильтрация данных

```
print(df[df['score'] > 80])
print(df[df['city'] == 'Алматы'])
print(df[(df['score'] > 80) & (df['attendance'] > 90)])
```

Фильтрация нужна тогда, когда мы хотим оставить в таблице только нужные записи.

В выражении df[df['score'] > 80] сначала создается логическое условие. Там, где условие истинно, строка остается. Где ложно - строка отбрасывается.

Если условий несколько, каждое условие обязательно берется в круглые скобки. Для операции И используется символ &, для ИЛИ - символ |.

7.9 Сортировка данных

```
print(df.sort_values('score'))
print(df.sort_values('score', ascending=False))
```

sort_values сортирует таблицу по указанному столбцу. По умолчанию сортировка идет по возрастанию. Если нужно по убыванию, используется параметр ascending=False.

7.10 Создание нового столбца

```
df['passed'] = df['score'] >= 75
df['bonus'] = df['score'] * 0.1
print(df)
```

- df['passed'] = df['score'] >= 75 создаст логический столбец: True, если студент набрал 75 и больше, и False в остальных случаях.
- df['bonus'] = df['score'] * 0.1 показывает, что новый столбец можно получить вычислением на основе уже существующего.

7.11 Базовая статистика

```
print(df['score'].mean())
print(df['score'].max())
print(df['score'].min())
print(df['city'].value_counts())
```

Выражение	Смысл
df['score'].mean()	Средний балл
df['score'].max()	Максимальный балл
df['score'].min()	Минимальный балл
df['city'].value_counts()	Сколько студентов из каждого города

7.12 Группировка через groupby

```
print(df.groupby('group')['score'].mean())
print(df.groupby('city')['score'].agg(['min', 'max', 'mean']))
```

groupby позволяет разбить таблицу на группы по значению столбца и выполнить вычисление внутри каждой группы.

В первом примере считается средний балл по каждой группе A, B и C.

Во втором примере сразу считаются минимум, максимум и среднее по каждому городу. Это уже маленький аналитический отчет.

8. Частые ошибки студентов

Ошибка	Почему возникает	Как исправить
Путают df["name"] и df[["name", "score"]]	Не различают один столбец и список столбцов	Для нескольких столбцов всегда передавайте список внутри двойных скобок
Пишут условия без скобок	Не знают, что при & и | каждая часть должна быть обособлена	Писать так: (условие1) & (условие2)
Забывают print или не смотрят на вывод	Запускают код механически, не анализируя результат	После каждой ячейки сравнивайте ожидание и реальный вывод
Сразу пишут большой блок кода	Теряется понимание, где возникла ошибка	Разбивайте код на небольшие ячейки и проверяйте каждую

9. Практическая работа на сегодняшнюю пару

9. Создайте DataFrame с учебным датасетом из этой методички.
10. Выведите первые 5 строк, список столбцов и размер таблицы.
11. Покажите только столбцы name и score.
12. Выведите первых 3 студентов через iloc.
13. Найдите всех студентов с баллом больше 80.
14. Найдите всех студентов из Алматы.
15. Найдите студентов, у которых score больше 80 и attendance больше 90.
16. Отсортируйте таблицу по score по возрастанию и по убыванию.
17. Создайте столбец passed.
18. Создайте столбец bonus.
19. Найдите средний, максимальный и минимальный балл.
20. Посчитайте количество студентов в каждом городе.
21. Найдите средний балл по группам.
22. Определите топ-3 студентов по score.
23. Сделайте 3 текстовых вывода по таблице своими словами.

10. Готовый итоговый пример для повторения дома

```
import numpy as np
import pandas as pd

# NumPy
arr = np.array([10, 20, 30, 40, 50])
print(arr)
print(arr.shape)
print(arr + 5)
print(arr.mean())

# Pandas
data = {
    'name': ['Алина', 'Руслан', 'Диана', 'Мадина', 'Ержан', 'Айгерим', 'Дамир',
'Нурсултан'],
    'age': [17, 18, 19, 18, 17, 20, 19, 18],
    'group': ['A', 'A', 'B', 'B', 'A', 'C', 'C', 'B'],
    'city': ['Алматы', 'Алматы', 'Астана', 'Астана', 'Шымкент', 'Алматы', 'Тараз',
'Алматы'],
```

```
    'score': [85, 72, 91, 88, 67, 95, 70, 83],
    'attendance': [90, 80, 95, 92, 75, 98, 85, 88]
}

df = pd.DataFrame(data)
print(df.head())
print(df[['name', 'score']])
print(df[df['score'] > 80])
print(df.sort_values('score', ascending=False))
print(df.groupby('group')['score'].mean())
```

Этот пример полезно переписать и запустить самостоятельно. После этого попробуйте изменить данные: добавить нового студента, поменять баллы, изменить города и посмотреть, как меняется результат анализа.

11. Домашняя работа

24. Повторить все примеры из разделов NumPy и Pandas в Google Colab.
25. Сделать собственную таблицу минимум из 8 записей на тему: студенты, фильмы, товары или игры.
26. В своей таблице выполнить минимум 8 операций: head, columns, выбор столбцов, iloc, фильтрация, сортировка, новый столбец, mean или groupby.
27. Написать 3-5 выводов по данным обычным текстом.

12. Короткий вывод

Сегодняшняя пара должна дать вам прочную связку: NumPy отвечает за массивы и быстрые числовые операции, а Pandas - за таблицы и анализ данных.

Если вы понимаете, как устроены массивы, индексы, столбцы, строки, фильтрация и сортировка, то дальше вам будет намного легче двигаться к анализу данных, визуализации, машинному обучению и AI.